

目录

常问经典面试题JAVA

方法重载和方法覆盖之间的区别

访问权限修饰符

抽象类和接口之间的区别

ArrayList和LinkedList之间的区别

HashMap在jdk7.0前后底层变化

TreeMap原理

常用线程池

数据库事务(特性、隔离问题、隔离级别)

MySQL索引失效

SQL语句优化

union和union all的区别

Get和Post的区别

Vue2数据劫持和数据代理

JavaWeb请求和响应有哪几部分组成

servlet生命周期

传统访问数据库方式

Git文件在三大区域穿梭

SpringMVC核心 组件

SpringMVC执行流程

日志

Spring AOP

Spring事务传播特性(七种)

SpringBoot自动配置

SpringBoot默认文件上传接口

Quartz核心API

RabbitMQ工作队列模式

Redis持久化策略

Redis主从模式

Redis哨兵模式

Redis分片集群

Dubbo

Nacos

SpringCloud中OpenFeign的使用

Sentinel

Spring Cloud Gateway

-----分割-----

双亲委派策略

SQL慢查询

MySQL中你都知道哪些索引

MySQL InnoDB引擎中**聚簇索引**与**非聚簇索引**的区别

什么OOP(面向对象编程)

如何解决OOP问题

Redis数据类型和使用场景

程序运行过程中偶尔出现SQL连接超时，过一会又正常

内存泄漏

微服务Gateway网关作用

Java中的缓存

MySQL回表操作

MySQL中水平分表和垂直分表

过滤器和拦截器区别

Java中大文件上传
反射机制核心类
Java创建对象的方式
线程池的创建和使用方式

☺ 常问经典面试题JAVA

🔗 方法重载和方法覆盖之间的区别

- 含义不同
 - 方法重载: Overload
 - 方法覆盖: Override
- 发生的位置不同
 - 方法重载: 需要发生在同一个类体中
 - 方法覆盖: 需要发生在有继承关系的两个类中
- 对参数的要求不同
 - 方法重载: 参数必须不同[类型 个数 顺序]
 - 方法覆盖: 参数必须相同
- 对返回类型的要求不同
 - 方法重载: 没有要求
 - 方法覆盖: 分版本
 - jdk5.0之前 一样
 - jdk5.0开始 协变返回类型
- 作用不同
 - 方法重载: 同时满足用户的不同需求
 - 方法覆盖: 继承方法重写

协变返回类型: 父类返回类型的子类

🔗 访问权限修饰符

访问权限修饰符:

- **public**: 公共的 谁都可以使用的
- **protected**: 受保护的 只有本包内可以访问 + 包外有继承关系的子类中可以访问
- **default**: 默认的 只有本包内可以访问
- **private**: 私有的 只有本类可以访问的

各自能修饰哪些东西?

修饰符	类	成员[属性+方法]
public	T	T
protected	F	T
default	T	T
private	F	T

- 面试题:
A类里面定义一个public修饰的方法 在B类里面能用吗?
不一定 如果B类想要访问A类里面的方法 前提是需要先访问A类

🔗 抽象类和接口之间的区别

抽象类: class 里面的属性都是静态属性 可以与抽象方法/普通方法
接口: interface 里面的属性都是静态属性 只能有抽象方法
jdk8.0开始 接口里面可以出现普通方法 -> static / default

🔗 ArrayList和LinkedList之间的区别

```
1 ArrayList底层基于数组实现的
2 优点: 查找效率高
3 缺点: 添加 删除元素效率低
```

```
1 LinkedList底层基于链表实现的
2 优点: 添加 删除元素效率高
3 缺点: 查找 遍历效率低
```

什么时候用 ArrayList?

🔴 适合场景

- 🔴 频繁读、很少写的场景，例如：
- 🔴 随机访问元素（比如根据索引访问）。

- 数据操作主要集中在列表的末尾（**尾部插入或删除操作**）。

- **典型场景**

- 缓存数据。
- 存储查询结果集。

什么时候用 LinkedList?

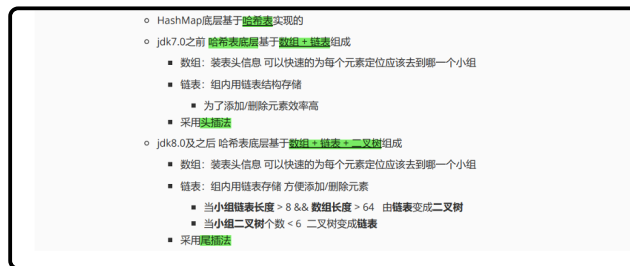
- **适合场景**

- 频繁写、很少读的场景，例如：
- **需要频繁插入或删除元素**，特别是操作链表的头尾时。
- **不需要随机访问**，只需要按顺序遍历。

- **典型场景**

- 队列、栈的实现。
- 存储实时更新的动态数据。

🔗 HashMap在jdk7.0前后底层变化



HashMap核心实现机制

- **哈希冲突解决**：采用链表 + 红黑树。
- **扩容机制**：当负载因子超出阈值时，`HashMap` 会进行 **2 倍扩容**。

ConcurrentHashMap 的详细解析

ConcurrentHashMap 是线程安全的哈希表实现，专为高并发设计，支持无锁或低锁的高效并发访问。

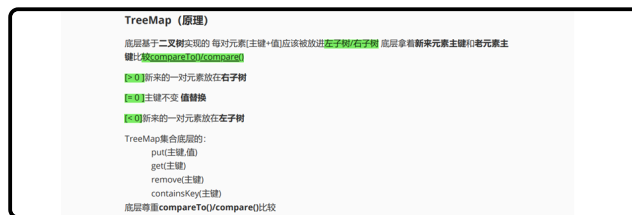
特性

- **线程安全**：支持多线程环境。
- **不支持 null 键和值**：插入 `null` 会抛出异常。
- **分段锁 + CAS 机制**：提供更高的并发性能。
- **存储无序**：数据存储顺序与插入顺序无关。

核心实现机制

- **分段锁（JDK 7）**：将数据分为若干段，每段独立加锁。
- **CAS + synchronized（JDK 8）**
 - CAS 处理简单更新操作。
 - synchronized 处理复杂操作（如扩容）。

TreeMap原理



常用线程池



数据库事务(特性、隔离问题、隔离级别)



MySQL索引失效



遵循最左前缀原则:

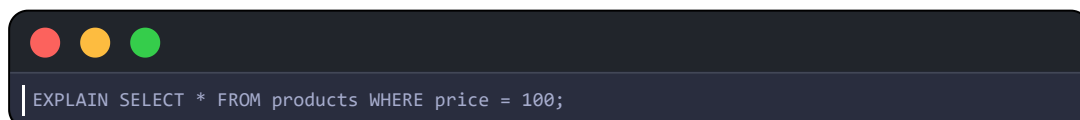
对于复合索引, 查询条件必须从索引的最左列开始, 否则索引可能失效。

例如, 索引是 (a, b, c), 查询条件必须包含 a, 才能有效使用索引。

🔗 如何排查索引效果?

使用 EXPLAIN 命令来查看 MySQL 的执行计划, 它能够展示 MySQL 是否使用了索引。

你可以在查询语句前加上 EXPLAIN 来查看具体的执行计划, 例如:



通过 EXPLAIN 的输出, 观察以下几个关键字段:

- **type** (访问类型)：显示查询使用的访问方法。如果是 `ALL`，说明是**全表扫描**；如果是 `index` 或 `range`，表示**使用了索引**。
- **key** (使用的索引)：显示使用的索引名。如果是 `NULL`，表示没有使用索引。
- **rows** (扫描的行数)：表示查询时扫描的行数，行数越少通常效率越高。

📖 SQL语句优化

sql语句优化:

1. 建议少用 * 代替列名
2. 用 `exists` 代替 `in`
3. 多表连接时，尽量减少表的查询次数
4. 删除全表数据的时候用 `truncate` 代替 `delete`
5. 合理使用索引 (详见索引)
6. sql语句尽量大写，Oracle 会默认把小写转换成大写在执行
7. 在保证语句完整的情况下，多使用 `commit(begin...end)`
8. 优化 `group by` 将不需要的数据尽量在分组之前过滤掉
9. 连表查询的时候尽量使用表的别名，减少解析时间
10. 表连接在 `where` 之前，`where` 条件过滤顺序，能够更多的过滤数据的数据放在前面

💡 数据库结构优化

✅ 选择合适的数据类型

- `VARCHAR` 适用于**变长**字符串，`CHAR` 适用于**定长**数据。
- `INT` 代替 `BIGINT`，节省存储空间。
- 使用 `TINYINT` 存储布尔值，代替 `BOOLEAN`。

✅ 分库分表

- **分库**：数据量大时，拆分数据库，减轻单库压力。
- **分表**：使用 **水平分表** 或 **垂直分表** 解决大表问题。

💡 MySQL 大表优化策略

当表的数据量过大（千万级以上）时，可以采用以下策略：

1. **分区表**：使用 MySQL `PARTITION` 特性，将表数据按照**时间**、**ID** 进行分区，提高查询效率。
2. **垂直拆分**：将表中访问频率较高的字段和较低的字段拆分到不同的表中，减少 I/O。
3. **冷热数据分离**：将历史数据存储到**归档表**中，提升主表查询效率。

📖 union和union all的区别

`union`：对两个结果集进行并集操作，不包括重复行，同时进行默认规则的排序；
`union all`：对两个结果集进行并集操作，包括重复行，不进行排序；

📖 Get和Post的区别

- **GET**
- 默认明文方式，速度快，毫无安全性，通过**浏览器地址栏传递值**，格式为？
`key=value&key2=value2&keyN=valueN`，不支持中文，最多传递 2000 个字符，链接提交肯定是 `get`
- **POST**
- 速度慢，安全性高，通过**请求的消体中的获得传递值**，我们可以通过浏览器的 f12 开发人员工具中的 **network 网络选项卡**查看传值情况，**支持中文，没有大小限制**，上传操作必须使用 `post`，**链接只能使用 get**，无法使用 `post`

Vue2数据劫持和数据代理

10. **Vue2实例对用户书写的data对象做了哪些处理**

1:数据劫持

Vue实例获取到用户书写的data对象之后,不管封装了几层都会对这个对象添加**可响应式功能**,对象的每个属性都被添加了**reactiveGetter()**和**reactiveSetter()**两个函数

如果data中的数据被读取,则执行reactiveGetter()

如果data中的数据被修改,则执行reactiveSetter(newVal)

这个函数中同时会修改页面模板中对应的数据,这就是**可响应式**

或者说**单向绑定**是如何实现的,这些被施加了可响应式功能的数据以及get和set函数都被封装在Vue实例的_data对象

这个操作被称之为**数据劫持**

2:数据代理

一个对象可以对另外一个对象中的属性进行操作,则称之为**数据代理**

Vue实例通过数据代理将_data中的数据代理到Vue实例的表层

这样做的好处是,我们在模板中使用指令或插值语法时,不需要每次都添加_data前缀,因为Vue实例表层数据都可以直接在模板中使用

JavaWeb请求和响应有哪几部分组成

5. JAVAWEB中请求和响应分别有哪几部分组成并举例说明?

请求:

- 请求方法: get/post/put/delete + 请求资源-/index.html + 协议版本-HTTP/1.1
- 请求头: Host Accept ...
- 消息体: 参数. key=value&key1=value1 ("key":"value")

响应:

- 状态行: 协议版本 + 状态码 + 字符串 HTTP/1.1 200 OK
- 响应头: content-type: text/html; charset=utf-8
- 消息正文

servlet生命周期

6. 从请求处理的角度简述Servlet的生命周期 (就是**一次请求的处理过程**)

服务器启动:

- 读取自身配置文件
- 读取管理器的其他工程配置文件,成功启动

客户端请求到达:

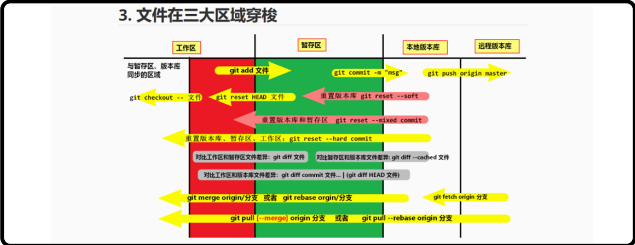
- tomcat接收请求,创建request对象和response对象.
- 根据请求判断静态资源还是动态资源,静态资源交给DefaultServlet
- 动态资源 根据url pattern,找到servlet-name,根据servlet-name找到servlet-class
- 判断内存中是否有对应Servlet类型的对象,如果有则返回,没有则使用**类反射创建Servlet对象**,【实例化】
- tomcat调用init完成初始化,并传入ServletConfig
- tomcat调用service执行请求
- Tomcat根据service中的response,打包后返回给客户端.
- 服务结束同时, tomcat调用destroy(),销毁

传统访问数据库方式

1. 传统的访问数据库的方式JDBC

- 加载驱动 Class.forName()
- 获得连接 Connection con..
- 创建执行SQL的Statement对象 PreparedStatement...
- 执行SQL返回结果 execute executeUpdate executeQuery()
- 处理结果 while(rs.next())...
- 关闭资源 close

Git文件在三大区域穿梭



SpringMVC核心 组件

什么是Web MVC?

1. 什么是Web MVC

- MVC是一种设计模式。它将应用程序分为模型、视图、控制器三部分

- 模型**：Java Bean
保存数据状态的Java Bean：实体类等
处理业务逻辑的Java Bean：Service、Dao
- 视图**：与用户交互界面
- 控制器**：实现了模型和视图的解耦
接收视图发送请求
调用**模型**处理业务
使用模型数据渲染视图，响应用户请求

什么是Spring MVC?

2. 什么是Spring MVC

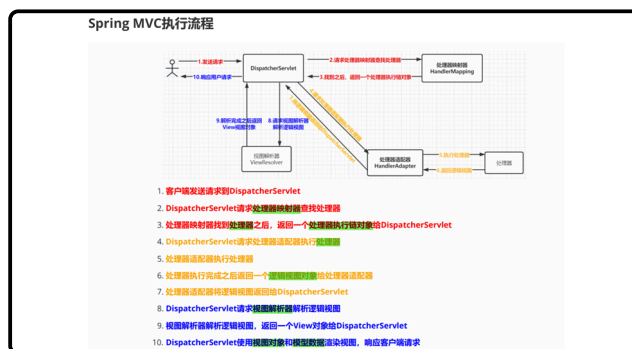
- Spring MVC是建立在Servlet API之上的**Web MVC**，从一开始就包含在**Spring MVC框架**中；
- 它的正式名称：**Spring Web MVC**（来自于它在Spring框架中的**模块名称**），通常被称为Spring MVC。

Spring MVC的核心组件

3. Spring MVC的核心组件

- 前端控制器DispatcherServlet**：拦截用户请求，将请求转发给其它组件，并负责页面渲染等，是Spring MVC的**核心控制器**；
- 处理器映射器HandlerMapping**：映射请求的URL到**处理器**（根据请求的URL查找处理器）
- 处理器适配器HandlerAdapter**：执行处理器（调用处理器器的方法）
- 视图解析器ViewResolver**：解析逻辑视图**ModelAndView**，创建**View对象**
- 视图**：开发来自定义等业务逻辑
- 数据绑定**，MultipartResolver，国际化组件....

SpringMVC执行流程



日志

日志

7. 使用SLF4J + Logback记录日志

- 为什么要记录日志？
方便查看线上代码的问题
记录用户的行为，方便审计用户行为
- 日志可以记录在哪里？
开发阶段一般输出到控制台

生产阶段一般输出到文件、数据库（不一定是关系型数据库，例如：ELK [noSQL]）

- 常用的日志框架有哪些？
门面日志框架（接口规范）：Apache Commons Logging、SLF4J
具体输出日志的框架：Log4j、JUL/Java Util Logging、Logback、Log4j2...
- 配置SLF4J + Logback
slf4j-api.jar、logback-core.jar、logback-classic.jar
- 日志级别
DEBUG < INFO < WARN < ERROR

Spring AOP

- AOP将应用程序分为**核心业务**和**非核心的公共服务**，AOP的关注点是系统中的非核心的公共服务；
- AOP可以通过**预编译**或者**运行动态代理**的方式，为横切多个对象的业务逻辑添加统一的功能（横切关注点：Cross Cutting Concern：事务、日志记录、权限检查、异常处理等）
- 使用AOP可以实现在不修改核心业务源代码的情况下为核心业务**添加统一的功能**，实现了核心业务和非核心的公共服务之间的**解耦**，提高了代码的**重用性**。

1. AOP 的核心思想

AOP 的目标是将**横切关注点**（例如日志、事务、权限验证等）从具体的业务逻辑中抽离出来。

这样做可以**避免在多个地方重复相同的代码**，提高代码的**可维护性和模块化**。

你可以将这些跨多个功能模块的代码独立为“切面”（Aspect），并在特定时机将它们“织入”业务逻辑中。

2. AOP 的关键概念

AOP 主要包括以下几个重要概念：

- 切面 (Aspect)**：切面**就是一个功能模块**，通常包含日志记录、事务管理、安全控制等功能，它将这些横切关注点从业务代码中分离出来。

切面可以包含多个**通知（Advice）**，定义了在某些何时进行特定的操作。

- **连接点（Join Point）**：连接点是程序执行过程中能够被切面的时机。
例如，一个方法的调用、抛出异常等。AOP 就是在这些时机点执行切面代码。
- **通知（Advice）**：通知是定义在连接点上的操作，也就是切面中要执行的具体功能。常见的通知类型包括：
 - **前置通知（Before）**：在方法执行前进行某些操作。
 - **后置通知（After）**：在方法执行后进行操作。
 - **环绕通知（Around）**：在方法执行前后都可以进行操作，甚至决定是否执行目标方法。
 - **异常通知（AfterThrowing）**：方法抛出异常时执行操作。
 - **返回通知（AfterReturning）**：方法正常返回时执行操作。
- **切入点（Pointcut）**：切入点是定义通知何时执行的规则，通常是一个表达式，指定在何种情况下应用通知。
比如，可以通过切入点匹配某个包下的所有方法。

切面

@Aspect

```
public class LoggingAspect {  
    @Before("execution(* com.example.service.*(..))")  
    public void logBefore() {  
        System.out.println("Logging before method execution");  
    }  
}
```

连接点

```
@Before("execution(* com.example.service.*(..))")  
public void logBeforeMethod(JoinPoint joinPoint) {  
    System.out.println("Method " + joinPoint.getSignature().getName() + " is being called");  
}
```

切入点

```
@Pointcut("execution(* com.example.service.*(..))")  
public void serviceMethods() {}
```

Spring AOP：基于**代理模式**，使用 JDK 动态代理或 CGLIB 生成代理类，**仅支持方法级别**的切入。

它适用于 Spring 管理的 Bean，动态织入，灵活性较高。

🔗 Spring事务传播特性(七种)

Spring事务传播特性（七种）

2.5.1 Spring的支持七种 **传播特性**

1. **REQUIRED**：表示如果当前存在一个事务，则加入该事务，否则将新建一个事务；**这个Spring事务都是以传播行为**
2. **REQUIRES_NEW**：表示不管是否存在事务，都创建一个新的事务，把原来的事务挂起。新的事务执行完毕，继续执行老的事务；
3. **SUPPORTS**：表示如果当前存在事务，就加入该事务；如果当前没有事务，**那就不能使用事务**；
4. **NOT_SUPPORTED**：表示不使用事务；如果当前存在事务，就把当前事务暂停，以事务方式执行；
5. **MANDATORY**：表示**必须在一个已有的事务中执行**。如果当前没有事务，**则抛出异常**；
6. **NEVER**：表示以**非事务方式**执行。如果当前存在事务，**则抛出异常**；
7. **NESTED**：这个是个**嵌套事务**。
如果当前存在事务，则在嵌套事务内执行；
如果当前不存在事务，则创建一个新的事务；
嵌套事务使用数据库中的**保存点**来实现，**即使嵌套事务回滚不影响外部事务，但外部事务回滚将导致嵌套事务回滚**；

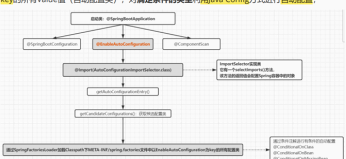
2.6 Spring事务的回滚规则

1. Spring事务默认只对运行时报错（非检查型）起作用（也就是出现运行时异常时，回滚数据）；
2. Spring事务默认对 非运行时异常（检查型异常、编译时常态）不起作用；
3. 可以使用 noRollbackFor 指定Spring事务对那些**异常**起作用；
4. 可以使用 rollbackFor 指定Spring事务对那些**异常**不起作用；

SpringBoot自动配置

4. Spring Boot的自动配置

1. Spring Boot自动配置主要是通过Spring工厂加载机制实现的；
2. 自动类上的@SpringBootApplication注解包含了三个注解：@SpringBootConfiguration、@EnableAutoConfiguration、@ComponentScan，其中最关键的注解就是@EnableAutoConfiguration；
3. @EnableAutoConfiguration注解导入一个AutoConfigurationImportSelector类型。这个类型实现了ImportSelector接口的selectImports()方法。在这个方法中使用Spring工厂加载机制加载classpath下META-INF文件类下的spring.factories文件中以EnableAutoConfiguration为key的所有Value值（自动配置类），为满足条件的类型利用Java Config方式行自动配置。



该图展示了Spring Boot自动配置的原理。它从@EnableAutoConfiguration开始，通过ImportSelector接口（具体为SpringBootAutoConfigurationImportSelector）调用selectImports方法。该方法会加载classpath下的META-INF/spring.factories文件，并从中获取所有以EnableAutoConfiguration为key的Value值。这些Value值代表了需要自动配置的类。最后，这些类会通过Java Config的方式被加载并配置。

SpringBoot默认文件上传接口

Spring Boot默认使用 `StandardServletMultipartResolver` 进行文件上传

默认文件的大小限制1MB

```
1 spring:
2   servlet:
3     multipart:
4       max-file-size: 2MB
```

Quartz核心API

3. Quartz的核心API

1. **Job接口**：创建一个任务，必须要实现Job接口
2. **JobDetail**：描述Job任务，定义Job的ID (name, group)
3. **Trigger**：任务执行规则（触发任务执行的规则 - 时间规则）
SimpleTrigger：定义任务何时执行、执行频次、何时停止等
CronTrigger：使用CRON表达式定义任务执行的时间规则
4. **Scheduler**：调度器，使用Trigger定义的规则执行任务
5. JobBuilder, TriggerBuilder, JobKey, TriggerKey....
6. JobExecutionContext

RabbitMQ工作队列模式

7. RabbitMQ工作队列模式



在使用消息系统时，一般情况下生产者往队列里插入数据的速度是比较快的。但是消费者消费数据往往涉及到一些业务逻辑处理导致速度跟不上生产者生产数据。因此，如果一个队列只有一个消费者的话，很容易导致大量的消息堆积在队列里。这时，就可以使用工作队列。这样一个队列可以有多个消费者同时消费数据。

当队列有多个消费者时，消息会被哪个消费者消费呢？这里主要有两种模式：

1. **轮询分发**：一个消费者消费一条，按均分配；
2. **公平分发**：根据消费者的消费能力进行公平分发，处理快的处理的多，处理慢的处理的少；按劳分配。

交换机类型：`fanout`、`direct`、`topic`、`headers`

Redis持久化策略

RDB持久化是Redis定期将内存中的数据写入磁盘。操作过程：主进程创建一个子进程，先将数据写入临时文件，写入成功后，再替换之前的文件，使用二进制压缩存储，整个过程快速不占用任何IO操作，保证Redis的高效性。

AOF持久化以日志的形式记录服务器所处理的每一个写操作和删除操作（以文本的方式记录，可以打开文件看到详细的操作记录）。

1. **如何开启aof持久化**
使用redis.conf中的 `appendonly yes` 设置
默认设置：`appendonly no`表示不开启aof持久化

混合持久化模式：结合 RDB 和 AOF 的优点

什么是混合持久化？

Redis 4.0 引入了混合持久化模式，通过将 RDB 和 AOF 结合：

1. RDB 保存全量数据快照。

2. **AOF** 记录 RDB 快照之后的增量操作。

混合持久化的文件格式为 AOF，前半部分是 RDB 快照数据，后半部分是 AOF 日志。

Redis主从模式

1.1 Redis的主从模式

Redis的**主从模式**，使用**异步复制**，从节点异步从主节点复制数据，主节点提供**读写服务**，从节点只提供**读服务**（这个是默认配置，可以通过修改配置文件 replica-read-only 控制）。主节点可以有多个从节点。

配置一个从节点只需要在redis.conf文件中指定：

```
replicaof master-ip master-port
```

1.1.1 主从复制配置

一主一从、一主多从、一主一从...

1.1.2 主从复制基本过程

1. Slave启动时，向Master发送**SYNC**命令，以实现**全量复制**。
2. Master(主库)接到SYNC请求，会在后台**保存快照**（也就是进行**RDB持久化**），并将快照期间接收到的命令缓存起来。
3. 快照完成后，Master(主库)会将快照文件和所有的缓存的命令发送给Slave(从库)。
4. Slave(从库)接收后，会载入快照文件并执行缓存的命令，从而完成复制的**初始化**。
5. 在数据库使用阶段，Master(主库)会**自动**把每次收到的命令**推送**到从服务器。

Redis哨兵模式

1.2 Redis哨兵模式

1. 主从方式有个问题，就是 Master(主库)挂了之后，**无法重新选举**新的节点作为主节点进行写操作，导致服务不可用。
2. Redis提供了**哨兵主库**来实现监控Redis系统的运行情况，能够实现如下功能：
 1. 监控主从数据库运行是否正常。
 2. 当主数据库出现故障时，自动将**从数据库**转换为**主数据库**。

1.2.1 哨兵配置

- 配置 /opt/redis/sentinel.conf 文件：

```
1 daemonize yes # 改为yes
2 logfile /opt/redis/sentinel.log
3 sentinel monitor mymaster 192.168.85.128 6379 1
4 sentinel down-after-milliseconds mymaster 5000 # 改为5s，默认30s
```

- 启动哨兵：redis-sentinel /opt/redis/sentinel.conf
- 进入哨兵：redis-cli -h 192.168.1.128 -p 26379
- 查看哨兵信息：redis-cli -h 192.168.1.128 -p 26379 info sentinel

Redis分片集群

- Redis**集群**是一个提供在**多个Redis节点间**共享数据的**逻辑库**。
- Redis集群并**不支持处理多个key的命令**（比如lset、mget、sdiff、sinter、union等），因为需要在不同的节点间**移动数据**，从而达到像Redis那样的性能，在高负载的情况下可能会导致不可预期的错误。
- Redis集群通过**分片**来提供一定程度的可用性，在实际环境中当**某个节点宕机**或者**不可达**的情况下继续处理命令。

1.3.1 Redis 集群的优势

- 自动**分片**数据到不同的节点上。
- 整个集群的部分节点失败或者不可达的情况下**能够继续处理命令**。

1.3.2 Redis 集群的数据分片

Redis集群引入了**哈希槽**的概念。

Redis 集群有 16384 个哈希槽（**2¹⁴**），每个key通过 **crc16**校验 后对16384取模来决定放置哪个槽，**集群的每个节点**负责一部分hash槽。

举个例子：比如当前集群有3个主节点，那么：

节点 A 包含 0 到 5500 号哈希槽。

节点 B 包含 5501 到 11000 号哈希槽。

节点 C 包含 11001 到 16383 号哈希槽。

这种结构很容易**添加或者删除节点**：比如我们想新添加个节点D，我们需要将节点 A、B、C 中的部分槽移动到D上。如果我想移除节点A，需要将A中的槽移到B和C节点上，然后将没有任何槽的A节点从集群中移除即可。由于**从一个节点将哈希槽移动到另一个节点并不会停止服务**，所以无论添加删除或者改变某个节点的哈希槽的数量都不会造成集群**不可用**的状态。

1.3.3 Redis集群安装

安装节点数：必须有3个或3个以上的主节点，所以，采用3主3从方式安装。

Dubbo

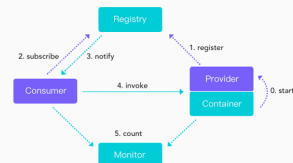
Apache Dubbo | 'dabau] 是一款**高性能、轻量级**的开源Java RPC框架，它提供了三大核心能力：**面向接口的远程方法调用**，**智能容错和负载均衡**，以及**服务自动注册和发现**。

Dubbo架构

1. 架构中的角色
 - Provider**：服务提供者
 - Consumer**：服务消费者
 - Registry**：服务注册和发现的中心
 - Monitor**：监控中心，用于统计服务调用情况
 - Container**：Dubbo容器

Dubbo Architecture

→ Init → async → sync



2. 调用关系
0. **消费者**向**生产者**、**注册**、**运行**服务提供者，消费者；
 1. **服务提供者**在启动时，向注册中心**注册**自己提供的服务；
 2. **服务提供者**在启动时，向注册中心**订阅**自己需要的服务；
 3. 注册中心返回服务提供者的**地址列表**给消费者；
如果有服务变更服务者的上线或下线，注册中心会基于**长连接的方式**推送变更给消费者；
 4. 服务消费者从地址列表中，基于**轮询负载均衡策略**，选择一个服务提供者进行调用。如果调用**失败**，可以尝试其它提供者；
 5. 服务消费者和服务提供者，在内存中**累计调用时间**和**调用次数**，每分钟向监控中心发送一次统计数据；

Nacos

一个更易于构建**云原生应用**的**动态服务发现**、**配置管理**和**服务管理平台**。

SpringCloud中OpenFeign的使用

在 Spring Cloud 中使用 OpenFeign，一般按照以下步骤进行：

1. 引入依赖

在项目 pom.xml 文件中添加 OpenFeign 相关依赖：

配置 >

```
<?xml version="1.0" encoding="UTF-8"?>
<dependencies>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-openfeign</artifactId>
    <version>2.12.0</version>
  </dependency>
</dependencies>
```

2. 启用 OpenFeign

在主启动类上添加 `@EnableFeignClients` 注解，启用 OpenFeign 功能。

3. 创建 Feign 客户端接口

定义一个接口，使用 `@FeignClient` 注解指定要调用的服务名称：

配置 >

```
@FeignClient(name = "user-service")
public interface UserFeignClient {

    @GetMapping("/api/users/{id}")
    User getUserById(@PathVariable("id") Long id);

    @PostMapping("/api/users")
    User create(@RequestBody User user);
}
```

4. 注入并使用 Feign 客户端

在需要使用的类中注入 Feign 客户端接口，并调用其方法：

配置 >

```
@Service
public class UserServiceImpl {

    private final UserFeignClient userFeignClient;

    @Autowired
    public UserServiceImpl(UserFeignClient userFeignClient) {
        this.userFeignClient = userFeignClient;
    }

    public User getUser(Long id) {
        return userFeignClient.getUserById(id);
    }

    public User create(User user) {
        return userFeignClient.createUser();
    }
}
```

例如，在一个微服务架构中，有 user-service 提供用户相关的接口，上述代码中的

UserFeignClient 就可以方便地调用 user-service 提供的接口。

通过 OpenFeign，可以实现服务之间的无缝通信，并且能够更好地管理和维护代码。

Sentinel

Sentinel是面向**分布式**、**多语言异构化服务架构**的**流量治理组件**，主要以**流量**为切入点，从**流量控制**、**流量路由**、**熔断降级**、**系统自适应保护**等多个维度来帮助用户保障微服务的稳定性。

Spring Cloud Gateway

Spring Cloud Gateway项目是一个基于Spring WebFlux开发的**API 网关**的框架，它的**目标**是为微服务架构提供一种**简单、有效的统一 API 路由管理方式**，且基于 Filter 链的方式提供了网关基本的功能，例如：安全、限流、熔断、路径重写、日志监控等功能。

3. Spring Cloud Gateway 功能、特点

- Built on Spring Framework 5, Project Reactor and Spring Boot 2.0
建立在Spring 5.0，Project Reactor (Spring WebFlux)，Spring Boot 2.0之上
- Able to match routes on any request attribute.
可以对所有的请求属性进行路由匹配
- Predicates and filters are specific to routes.
可以对特定路由进行断言和过滤
- Circuit Breaker integration.
集成断路器（微服务保护【限流、熔断、降级】）
- Spring Cloud DiscoveryClient integration
集成服务发现
- Easy to write Predicates and Filters
容易编写断言和过滤器（通过配置文件编写、通过代码编写）
- Request Rate Limiting
请求限流
- Path Rewriting
路径重写

📁 双亲委派策略

1. 定义：

- 双亲委派策略是 Java 类加载器（ClassLoader）加载类的一种机制。
- 当一个类加载器收到加载类的请求时，它首先会委托给父类加载器去加载，只有父类加载器无法加载时，自己才会尝试加载。

2. 加载顺序：

- **自底向上**：从子类加载器向父类加载器逐级委托。
- **自顶向下**：从父类加载器向子类加载器逐级尝试加载。

3. 优点：

- **避免重复加载**：确保一个类只会被加载一次，防止内存中出现多个相同的类。
- **安全性**：防止核心类库（如 `java.lang` 包）被篡改，确保 Java 核心 API 的稳定性。

📁 SQL慢查询

1. **开启慢查询日志**：通过设置 `slow_query_log` 和 `long_query_time` 参数，记录执行时间超过指定阈值的SQL语句，帮助定位慢查询。

2. **分析慢查询日志**：使用 `mysqldumpslow` 或 `pt-query-digest` 等工具分析日志，找出最耗时的查询。

3. **使用EXPLAIN**：通过 `EXPLAIN` 命令分析查询的执行计划，查看是否使用了索引、扫描行数等信息，判断是否存在性能瓶颈。

重要字段解析：

- `type`：访问类型，性能从高到低依次为：`ALL`（**全表扫描**）> `index`（索引扫描）> `range`（索引范围扫描）> `ref`（非唯一索引扫描）> `eq_ref`（唯一索引扫描）。
- `key`：显示使用的索引名。如果是 `NULL`，表示没有使用索引。
- `rows`：表示查询时扫描的行数，行数越少通常效率越高。

4. 优化查询：

- **添加索引**：确保查询条件中的字段有合适的索引，尤其是WHERE、JOIN、ORDER BY等子句中的字段。
- **重写查询**：简化复杂查询，避免子查询、嵌套查询，或使用JOIN替代。
- **避免全表扫描**：确保查询条件能够有效利用索引，减少全表扫描。

5. 优化表结构：

- **规范化与反规范化**：根据业务需求，适当进行规范化或反规范化设计。
- **分区表**：对大表进行分区，提升查询性能。

📁 MySQL中你都知道哪些索引

1. B+树索引：

- 最常见的索引类型，适用于等值查询、范围查询和排序操作。
- InnoDB和MyISAM存储引擎默认使用B+树索引。

2. 全文索引：

- 用于全文搜索，支持对文本内容进行关键词检索。
- 适用于 `MATCH AGAINST` 操作，常用于大段文本的搜索。

3. 空间索引（R树索引）：

- 用于地理空间数据（如经纬度），支持空间查询（如距离、范围等）。
- 使用R树实现，适用于GIS数据类型。

4. 前缀索引：

- 对字符串列的前缀部分建立索引，节省存储空间。
- 适用于较长的字符串列，但可能会影响查询精度。

5. 组合索引（复合索引）：

- 对多个列组合建立索引，遵循最左前缀原则。
- 适合多条件查询，但需要注意列的顺序。

6. 唯一索引：

- 确保索引列的值唯一，常用于主键或唯一约束。
- 可以加速查询并保证数据唯一性。

7. 主键索引：

- 特殊的唯一索引，不允许NULL值。
- 每个表只能有一个主键索引，通常与聚簇索引结合使用。

🔗 MySQL InnoDB引擎中聚簇索引与非聚簇索引的区别

在 MySQL 的 **InnoDB** 引擎中，**聚簇索引**和**非聚簇索引**的区别在于数据存储方式：

- **聚簇索引**：数据直接存储在索引结构中，查询时可以更快获取数据。
- **非聚簇索引**：只存储索引列和主键值，需要通过主键再查一次数据，所以速度稍慢。

每个表只能有一个 **聚簇索引**，但可以有多多个 **非聚簇索引**。

🔗 什么OOP(面向对象编程)

OOP（Object-Oriented Programming，面向对象编程）是一种编程范式，通过**对象**来组织代码和数据。

它的核心思想是将现实世界的事物抽象为程序中的对象，对象包含**属性**（数据）和**行为**（方法）。

OOP的四大特性是：

1. 封装（Encapsulation）：

将数据和方法封装在对象内部，隐藏实现细节，只暴露必要的接口。

2. 继承（Inheritance）：

子类可以继承父类的属性和方法，实现代码复用和层次化设计。

3. 多态（Polymorphism）：

同一个方法在不同对象中有不同的实现，分为编译时多态（重载）和运行时多态（重写）。

4. 抽象（Abstraction）：

通过抽象类和接口定义通用的行为和属性，隐藏复杂性。

📖 如何解决OOP问题

1. 设计良好的类结构：

- 根据业务需求，合理设计类和对象，确保**职责单一**（单一职责原则）。
- 使用**继承**和**多态**实现代码复用和扩展性。

2. 遵循设计原则：

○ SOLID原则：

- **S**：单一职责原则（Single Responsibility Principle）。
- **O**：开闭原则（Open/Closed Principle）。
- **L**：里氏替换原则（Liskov Substitution Principle）。
- **I**：接口隔离原则（Interface Segregation Principle）。
- **D**：依赖倒置原则（Dependency Inversion Principle）。

3. 使用设计模式：

- 针对常见问题，使用设计模式（如工厂模式、单例模式、观察者模式等）提供解决方案。

📖 Redis数据类型和使用场景

1. String（字符串）：

- **作用**：存储文本、数字或二进制数据。
- **使用场景**：
 - 缓存简单数据（如用户信息、配置项）。
 - 计数器（如文章阅读量、点赞数）。
 - 分布式锁（通过 `SETNX` 实现）。

2. Hash（哈希）：

- **作用**：存储键值对集合，适合存储对象。
- **使用场景**：
 - 存储对象属性（如用户信息：`name`、`age`、`email`）。
 - 分组数据（如商品分类下的商品详情）。

3. List（列表）：

- **作用**：按插入顺序存储字符串元素，支持双向操作。
- **使用场景**：
 - 消息队列（如任务队列、日志记录）。
 - 最新消息列表（如朋友圈动态、新闻推送）。

4. Set（集合）：

- **作用**：存储不重复的无序字符串集合。
- **使用场景**：

- 去重数据（如用户标签、好友关系）。
- 交集、并集、差集运算（如共同好友、推荐系统）。

5. Sorted Set（有序集合）：

- **作用：** 存储不重复的字符串集合，并按分数排序。
- **使用场景：**
 - 排行榜（如游戏积分、热搜榜）。
 - 范围查询（如按时间范围获取数据）。

6. Bitmap（位图）：

- **作用：** 通过位操作存储布尔值。
- **使用场景：**
 - 用户签到、活跃度统计。
 - 布隆过滤器（用于去重）。

🔗 程序运行过程中偶尔出现SQL连接超时，过一会又正常

可能的原因：

1. 数据库连接池问题：

- 连接池中的连接被耗尽，导致新请求无法获取连接。
- 连接池配置不合理（如最大连接数过小）。

2. 数据库性能瓶颈：

- 数据库负载过高，导致响应变慢。
- SQL查询性能差，执行时间过长。

3. 网络问题：

- 网络不稳定，导致连接超时。
- 数据库服务器与应用程序之间的网络延迟较高。

4. 数据库连接泄漏：

- 程序中未正确关闭数据库连接，导致连接池中的连接被耗尽。

5. 数据库配置问题：

- 数据库的连接超时时间设置过短。
- 数据库的最大连接数限制过低。

解决方法：

1. 优化连接池配置：

- 增加连接池的最大连接数（如 `maxActive`）。
- 设置合理的连接超时时间（如 `maxWait`）。
- 使用连接池的健康检查机制，定期检测连接的有效性。

2. 优化数据库性能：

- 分析慢查询日志，优化SQL语句。
- 添加索引，减少全表扫描。
- 对数据库进行分库分表，减轻单库压力。

3. 检查网络状况：

- 确保数据库服务器与应用程序之间的网络稳定。
- 使用网络监控工具排查网络延迟或丢包问题。

4. 修复连接泄漏：

- 确保每次使用完数据库连接后，正确关闭连接（如使用 `try-with-resources` 或 `finally` 块）。
- 使用连接池的泄漏检测功能（如Druid的 `removeAbandoned`）。

5. 调整数据库配置：

- 增加数据库的最大连接数。
- 调整数据库的连接超时时间（如 `wait_timeout`、`interactive_timeout`）。

6. 监控与报警：

- 使用监控工具（如Prometheus、Grafana）实时监控数据库和连接池的状态。
- 设置报警规则，及时发现和处理问题。

内存泄漏

什么是内存泄露？

内存泄露是指**程序中已分配的内存未能被正确释放，导致内存占用持续增加，最终可能引发 `OutOfMemoryError`**。

在Java中，虽然垃圾回收器（GC）会自动回收无用对象，但如果对象被无意中持有引用，GC无法回收这些对象，就会导致内存泄露。

内存泄露的常见原因：

1. 静态集合类：

- 静态集合（如 `static List`、`static Map`）持有对象的引用，导致对象无法被回收。

2. 未关闭的资源：

- 未关闭的文件流、数据库连接、网络连接等，导致资源占用无法释放。

3. 监听器和回调：

- 注册了监听器或回调但未取消注册，导致对象无法被回收。

4. 内部类持有外部类引用：

- **非静态内部类**隐式持有外部类的引用，如果内部类对象未释放，外部类对象也无法被回收。

5. 缓存未清理：

- 缓存中的对象长期未被使用，但未及时清理，导致内存占用增加。

如何检测内存泄露？

1. 使用工具分析：

- **JVisualVM**：监控堆内存使用情况，分析对象占用。

- **MAT (Memory Analyzer Tool)**: 分析堆转储文件 (Heap Dump), 找出内存泄露的根源。

- **JProfiler**: 实时监控内存使用, 定位泄露点。

2. 查看GC日志:

- 启用GC日志 (如 `-Xloggc:gc.log`), 分析内存回收情况。

3. 堆转储分析:

- 使用 `jmap` 生成堆转储文件 (如 `jmap -dump:format=b,file=heap.bin <pid>`), 然后用工具分析。

如何解决内存泄露?

1. 及时释放资源:

- 使用 `try-with-resources` 或 `finally` 块确保资源 (如文件流、数据库连接) 被正确关闭。

2. 避免静态集合类滥用:

- 使用弱引用 (`WeakReference`) 或软引用 (`SoftReference`) 来存储对象。

- 定期清理不再使用的对象。

3. 取消监听器和回调:

- 在对象销毁时, 取消注册监听器或回调。

4. 使用合适的缓存策略:

- 使用LRU (**最近最少使用**) 等缓存淘汰策略, 定期清理缓存。

- 使用 `WeakHashMap` 或第三方缓存库 (如Guava Cache、Ehcache)。

5. 优化代码设计:

- 避免非静态内部类隐式持有外部类引用, 改用**静态内部类**或**弱引用**。

🏡 微服务Gateway网关作用

Spring Cloud Gateway的作用:

Spring Cloud Gateway是Spring Cloud生态系统中的API网关,

主要用于**统一管理微服务架构中的请求路由、过滤和负载均衡**。

它的核心作用包括:

1. 请求路由:

- 根据请求路径、请求头、请求参数等条件, 将请求转发到对应的微服务。

- 支持动态路由配置, 无需重启服务即可更新路由规则。

2. 负载均衡:

- 集成Ribbon或Spring Cloud LoadBalancer, 实现客户端负载均衡。

- 将请求分发到多个服务实例, 提高系统的可用性和性能。

3. 请求过滤:

- 提供全局过滤器 (Global Filter) 和网关过滤器 (Gateway Filter), 用于在请求转发前后执行逻辑 (如鉴权、日志记录、限流等)。

- 支持自定义过滤器, 满足特定业务需求。

4. 限流与熔断:

- 集成Resilience4j或Hystrix, 实现限流和熔断功能, 保护后端服务不被过载请求击垮。

5. 安全控制:

- 支持OAuth2、JWT等认证机制, 统一管理微服务的访问权限。
- 提供IP黑白名单、跨域 (CORS) 支持等功能。

6. 日志与监控:

- 记录请求和响应的日志, 便于问题排查和性能分析。
- 集成Micrometer, 支持Prometheus等监控工具。

7. 协议转换:

- 支持HTTP、WebSocket等协议的转换和适配。

核心组件:

- Route (路由):** 定义请求的转发规则, 包括目标URI、断言和过滤器。
- Predicate (断言):** 匹配请求的条件 (如路径、请求头、请求方法等)。
- Filter (过滤器):** 在请求转发前后执行逻辑 (如修改请求头、添加参数等)。

使用场景:

- 微服务入口:** 作为所有微服务的统一入口, 对外隐藏内部服务细节。
- 权限校验:** 在网关层统一进行身份认证和权限校验。
- 流量控制:** 通过限流和熔断保护后端服务。
- 日志记录:** 集中记录请求和响应日志, 便于监控和分析。
- 跨域支持:** 统一处理跨域请求, 避免每个服务单独配置。

示例配置:

```
spring:
  cloud:
    gateway:
      routes:
        - id: user-service
          uri: lb://user-service
          predicates:
            - Path=/user/**
          filters:
            - AddRequestHeader=X-Request-Id, 12345
```

总结: Spring Cloud Gateway作为微服务架构中的API网关,

核心作用是**请求路由、负载均衡、请求过滤、限流熔断、安全控制**等。它简化了微服务架构的复杂性, 提升了系统的可维护性和安全性。

什么是缓存？

缓存是一种**临时存储机制**，用于保存频繁访问的数据，以减少对**慢速存储**（如数据库、文件系统）的访问，从而提高系统性能。

Java中缓存的实现方式：

1. 本地缓存：

- **HashMap**：最简单的缓存实现，但缺乏过期策略和容量限制。
- **Guava Cache**：Google提供的缓存库，支持过期时间、最大容量、缓存淘汰策略（如LRU）。
- **Caffeine**：高性能缓存库，功能类似Guava Cache，但性能更好。
- **Ehcache**：**成熟的本地缓存框架**，支持分布式缓存。

2. 分布式缓存：

- **Redis**：基于内存的键值存储，支持多种数据结构（如String、Hash、List等），常用于分布式缓存。
- **Memcached**：高性能分布式内存缓存系统，适合存储简单的键值对。
- **Hazelcast**：分布式内存数据网格，支持分布式缓存和计算。

3. JVM缓存：

- **堆内缓存**：缓存数据存储在JVM堆内存中，访问速度快，但受限于堆内存大小。
- **堆外缓存**：缓存数据存储在JVM堆外内存中，**适合存储大量数据**（如Netty的ByteBuf）。

缓存的使用场景：

1. 数据库查询缓存：

- 缓存频繁查询的数据库结果，减少数据库压力。
- 例如：用户信息、商品详情。

2. 页面缓存：

- 缓存动态生成的页面或页面片段，减少服务器渲染开销。
- 例如：电商网站的商品列表页。

3. API响应缓存：

- 缓存API的响应结果，减少重复计算和外部服务调用。
- 例如：天气数据、汇率信息。

4. 会话缓存：

- 缓存用户会话信息，减少对数据库或文件系统的访问。
- 例如：用户登录状态、购物车数据。

5. 计算缓存：

- 缓存复杂计算结果，避免重复计算。
- 例如：推荐算法结果、统计报表。

缓存的常见问题及解决方案：

1. 缓存穿透：

- 问题：查询**不存在的数据**，导致请求直接落到数据库。
- 解决：使用布隆过滤器（Bloom Filter）或**缓存空值**。

2. 缓存雪崩：

- 问题：大量缓存**同时失效**，导致数据库压力骤增。
- 解决：设置缓存过期时间的随机值，避免同时失效。

3. 缓存击穿：

- 问题：**热点数据**失效后，大量请求直接落到数据库。
- 解决：使用**互斥锁**（Mutex Lock）或**永不过期策略**。

4. 数据一致性：

- 问题：缓存与数据库数据不一致。
- 解决：使用**双写策略**（先写数据库，再写缓存）或**缓存失效策略**。

示例代码（Guava Cache）：

```
import com.google.common.cache.Cache;
import com.google.common.cache.CacheBuilder;

public class GuavaCacheExample {
    public static void main(String[] args) {
        // 创建缓存
        Cache<String, String> cache = CacheBuilder.newBuilder()
            .maximumSize(100) // 最大容量
            .expireAfterWrite(10, TimeUnit.MINUTES) // 过期时间
            .build();

        // 写入缓存
        cache.put("key1", "value1");

        // 读取缓存
        String value = cache.getIfPresent("key1");
        System.out.println(value); // 输出: value1
    }
}
```

总结：Java中的缓存主要用于提高系统性能，常见的实现方式包括**本地缓存**（如Guava Cache、Caffeine）和**分布式缓存**（如Redis、Memcached）。

使用缓存时需要注意**缓存穿透、雪崩、击穿和数据一致性**等问题。

MySQL回表操作

定义

回表是指在使用**非聚簇索引**查询时，数据库无法直接拿到所有的列数据，因为非聚簇索引只存储了索引字段和主键，而没有存储完整的行数据。

因此，在查询时，数据库需要再通过**主键去聚簇索引**查找完整的行数据，产生一次额外的查询操作。这个额外的查询操作就叫做“回表”。

优化

回表不仅增加了查询的开销，还可能引发**随机 I/O**，影响性能。

通过合理的查询优化、索引设计和覆盖索引，我们可以有效减少回表操作，提高查询效率。

MySQL中水平分表和垂直分表

水平分表：

1. 定义：

- 水平分表是指**将一张表的数据按行拆分，存储到多个结构相同的表中。**
- 每个表存储一部分数据，所有表的表结构相同。

2. 实现方式：

- 按某个规则（如ID范围、哈希值、时间范围）将数据分散到多个表中。
- 例如：用户表按用户ID的哈希值分表，分成 `user_0`、`user_1`、`user_2` 等。

3. 使用场景：

- 单表数据量过大，导致查询和写入性能下降。
- 需要分散数据存储压力，提高系统扩展性。

4. 优点：

- 分散单表数据量，提升查询和写入性能。
- 支持水平扩展，适合大数据量场景。

5. 缺点：

- 查询时需要跨多个表，复杂度增加。
- 数据迁移和扩容较复杂。

垂直分表：

1. 定义：

- 垂直分表是指**将一张表按列拆分，存储到多个表中。**
- 每个表存储一部分列，所有表通过主键关联。

2. 实现方式：

- 将**常用列**和**不常用列**分开存储。
- 例如：用户表拆分为 `user_basic`（存储用户名、密码）和 `user_detail`（存储地址、电话）。

3. 使用场景：

- 表中包含大量不常用字段，导致单行数据过大。
- 需要优化查询性能，减少I/O开销。

4. 优点：

- 减少单行数据大小，提升查询效率。
- 按需加载数据，减少资源浪费。

5. 缺点：

- 查询时需要关联多个表，增加复杂度。
- 事务一致性维护较复杂。

📁 过滤器和拦截器区别

特性	过滤器（Filter）	拦截器（Interceptor）
规范	Servlet规范	Spring框架
工作范围	所有请求（包括静态资源）	仅Controller请求
实现方式	实现 javax.servlet.Filter 接口	实现 HandlerInterceptor 接口
执行时机	在Servlet之前或之后执行	在Controller方法执行前后执行
依赖	依赖Servlet容器（如Tomcat）	依赖Spring容器
使用场景	字符编码、全局权限校验、日志记录	登录验证、权限检查、性能监控

总结：

- **过滤器**：基于Servlet规范，拦截所有请求，适合**全局处理**（如字符编码、权限校验）。
- **拦截器**：基于Spring框架，仅拦截Controller请求，适合**业务逻辑处理**（如登录验证、性能监控）。

📁 Java中大文件上下传

大文件上传：

1. 分片上传：

- 将大文件分割成多个小文件（分片），逐个上传到服务器。
- 服务器接收分片后，按顺序合并成完整文件。
- **优点**：支持断点续传、减少单次上传压力。

2. 断点续传：

- 记录已上传的分片信息，上传中断后可以从断点继续上传。
- 使用**文件MD5**或**唯一标识**校验文件完整性。

3. 使用流式上传：

- 使用 `InputStream` 读取文件，通过HTTP或FTP等协议流式上传。
- 避免将整个文件加载到内存，减少内存占用。

4. 多线程上传：

- 使用多线程同时上传多个分片，提高上传速度。

大文件下载：

1. 分块下载：

- 将大文件分成多个块，客户端逐个下载并合并。
- 使用HTTP的 `Range` 头实现分块下载。

2. 断点续传：

- 记录已下载的文件大小，下载中断后可以从断点继续下载。

3. 使用流式下载：

- 使用 `OutputStream` 将文件流式写入本地，避免一次性加载到内存。

4. 多线程下载：

- 使用多线程同时下载文件的不同部分，提高下载速度。

🔗 反射机制核心类

Java反射机制的核心类位于 `java.lang.reflect` 包中，主要包括以下几个类：

1. `Class`：

- 表示一个类或接口的类型信息。
- 通过 `Class` 对象可以获取类的构造方法、方法、字段等信息。
- 获取 `Class` 对象的方式：
 - `Class.forName("全限定类名")`。
 - `对象.getClass()`。
 - `类名.class`。

2. `Constructor`：

- 表示类的构造方法。
- 通过 `Constructor` 对象可以创建类的实例。
- 示例：`Class.getDeclaredConstructor(参数类型)`。

3. `Method`：

- 表示类的方法。
- 通过 `Method` 对象可以调用类的方法。
- 示例：`Class.getDeclaredMethod("方法名", 参数类型)`。

4. `Field`：

- 表示类的字段（成员变量）。
- 通过 `Field` 对象可以获取或设置字段的值。
- 示例：`Class.getDeclaredField("字段名")`。

5. `Modifier`：

- 提供对类、方法、字段的修饰符（如 `public`、`private`、`static` 等）的解析。
- 示例：`Modifier.isPublic(method.getModifiers())`。

6. `Array`：

- 提供动态创建和操作数组的方法。
- 示例：`Array.newInstance(Class<?> componentType, int length)`。

反射的基本使用步骤：

1. 获取 `Class` 对象。
2. 通过 `Class` 对象获取 `Constructor`、`Method`、`Field` 等对象。
3. 设置访问权限（如 `setAccessible(true)`）。

4. 调用方法或操作字段。

反射的应用场景：

1. **动态加载类**：在运行时加载类并创建对象。
2. **框架开发**：如Spring通过反射创建Bean并注入依赖。
3. **注解处理**：通过反射读取注解信息。
4. **测试工具**：如JUnit通过反射调用测试方法。
5. **通用工具**：如序列化、反序列化工具。

总结：

- 反射机制的核心类包括 `Class`、`Constructor`、`Method`、`Field`、`Modifier` 和 `Array`。
- 反射的主要作用是动态获取类的信息并操作类的成员，常用于框架开发、注解处理等场景。

📦 Java创建对象的方式

Java中创建对象的五种方式：

1. 使用 `new` 关键字：

- 最常见的方式，直接调用类的构造方法创建对象。

```
User user = new User();
```

2. 使用 `Class.newInstance()`：

- 通过反射调用类的**无参构造方法**创建对象。

```
Class<?> clazz = Class.forName("com.example.User");
User user = (User) clazz.newInstance();
```

3. 使用 `Constructor.newInstance()`：

- 通过反射调用类的**带参构造方法**创建对象。

```
Class<?> clazz = Class.forName("com.example.User");
Constructor<?> constructor = clazz.getConstructor(String.class, int.class);
User user = (User) constructor.newInstance("Alice", 25);
```

4. 使用 `clone()` 方法：

- 通过实现 `Cloneable` 接口并重写 `clone()` 方法**创建对象的副本**。


```
class User implements Cloneable {
    private String name;
    public User(String name) {
        this.name = name;
    }
    @Override
    protected Object clone() throws CloneNotSupportedException {
        return super.clone();
    }
}

User user1 = new User("Alice");
User user2 = (User) user1.clone();
```

5. 使用反序列化：

- 通过 `ObjectInputStream` 从文件或网络中读取字节流并反序列化为对象。

```
class User implements Serializable {
    private String name;
    public User(String name) {
        this.name = name;
    }
}

// 序列化
try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("user.ser"))) {
    oos.writeObject(new User("Alice"));
}

// 反序列化
try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream("user.ser"))) {
    User user = (User) ois.readObject();
}
}
```

其他创建对象的方式：

1. 工厂模式：

- 通过工厂类创建对象，隐藏对象创建的细节。

```
class UserFactory {
    public static User createUser() {
        return new User();
    }
}

User user = UserFactory.createUser();
```

2. 依赖注入 (DI)：

- 通过Spring等框架的**依赖注入机制**创建对象。

```
@Autowired
private User user;
```

3. Lambda表达式：

- 通过函数式接口创建对象。

```
Runnable runnable = () -> System.out.println("Hello");
```

总结：

Java中创建对象的方式主要有：

1. `new` 关键字。
2. `Class.newInstance()` 。
3. `Constructor.newInstance()` 。
4. `clone()` 方法。
5. 反序列化。

此外，还可以通过**工厂模式**、**依赖注入**、**Lambda表达式**等方式创建对象。每种方式适用于不同的场景，选择合适的创建方式可以提高代码的灵活性和可维护性。

🧵 线程池的创建和使用方式

线程池的创建：

Java中通过 `java.util.concurrent.Executors` 工具类或直接使用 `ThreadPoolExecutor` 类创建线程池。

1. 使用 `Executors` 工具类：

`Executors` 提供了几种常见的线程池创建方法：

○ `newFixedThreadPool`：

- 创建**固定大小**的线程池。

```
ExecutorService executor = Executors.newFixedThreadPool(5);
```

○ `newCachedThreadPool`：

- 创建可缓存的线程池，**线程数根据任务数量动态调整**。

```
ExecutorService executor = Executors.newCachedThreadPool();
```

○ `newSingleThreadExecutor`：

- 创建单线程的线程池，**保证任务按顺序执行**。

```
ExecutorService executor = Executors.newSingleThreadExecutor();
```

○ `newScheduledThreadPool`：

- 创建支持**定时及周期性**任务执行的线程池。



```
ScheduledExecutorService executor = Executors.newScheduledThreadPool(3);
```

2. 使用 `ThreadPoolExecutor` 类：

`ThreadPoolExecutor` 提供了更灵活的线程池配置，可以自定义核心线程数、最大线程数、任务队列等参数。



```
int corePoolSize = 5; // 核心线程数
int maxPoolSize = 10; // 最大线程数
long keepAliveTime = 60; // 空闲线程存活时间
TimeUnit unit = TimeUnit.SECONDS; // 时间单位
BlockingQueue<Runnable> workQueue = new LinkedBlockingQueue<>(100); // 任务队列

ThreadPoolExecutor executor = new ThreadPoolExecutor(
    corePoolSize, maxPoolSize, keepAliveTime, unit, workQueue
);
```

线程池的使用：

1. 提交任务：

- 使用 `execute()` 方法提交任务（**无返回值**）。



```
executor.execute(() -> System.out.println("Task executed"));
```

- 使用 `submit()` 方法提交任务（返回 `Future` 对象，**可获取任务执行结果**）。



```
Future<String> future = executor.submit(() -> "Task result");
String result = future.get(); // 获取任务结果
```

2. 关闭线程池：

- 使用 `shutdown()` 方法平滑关闭线程池（等待已提交任务执行完成）。**排队中的任务可以执行完**



```
executor.shutdown();
```

- 使用 `shutdownNow()` 方法立即关闭线程池（尝试中断正在执行的任务）。**退回排队中的任务**



```
executor.shutdownNow();
```

3. 监控线程池状态：

- 通过 `ThreadPoolExecutor` 的方法获取线程池状态：

- `getPoolSize()`：当前线程池中的线程数。
- `getActiveCount()`：正在执行任务的线程数。
- `getCompletedTaskCount()`：已完成的任务数。

总结：

- Java中通过 `Executors` 工具类或 `ThreadPoolExecutor` 类创建线程池。
- 线程池的核心作用是复用线程、提高资源利用率和任务执行效率。

- 使用线程池时需合理配置核心参数，并根据任务类型选择合适的线程池类型。